

제138회 정보관리기술사 해설집

2026.02.07

국가기술자격 기술사 시험문제

기술사 제 138 회

제 2 교시 (시험시간: 100 분)

분야	정보통신	자격종목	정보관리기술사	수검 번호		성 명	
----	------	------	---------	----------	--	--------	--

※ 다음 문제 중 4 문제를 선택하여 설명하시오. (각 10 점)

1. 최근 상용 DBMS 를 오픈소스 DBMS 로 전환하는 수요가 늘고 있다. 다음 내용을 설명하시오.

가. 오픈소스 DBMS 전환 배경

나. 오픈소스 DBMS 전환 시 검토해야 할 제약사항

다. 오픈소스 DBMS 전환 시 단계별 마이그레이션 절차

라. 신뢰성 확보를 위한 고가용성(HA) 아키텍처 구성 방안

2. 다음 내용을 설명하시오.

가. ISP(Information Strategy Planning)의 정의 및 목적

나. ISP 수행방법론 체계와 절차 (메모: 환경분석 통합)

다. ISP, ISMP(Information System Master Plan) 비교

3. 인공지능 학습에 이용하는 GPU(Graphics Processing Unit)와 TPU(Tensor Processing Unit)에 대하여 다음 내용을 설명하시오.

가. GPU 와 TPU 개념 (메모: CUDA, TensorFlow)

나. GPU 와 TPU 비교

다. 최근 GPU 보다 TPU 를 사용하는 이유

라. 효율성 측면에서 TPU 의 장점 및 향후 전망

4. B 기관은 기존에 운영 중이던 대국민 서비스 포털, 통합 업무 시스템, ERP 및 통합 인허가 관리 시스템을 클라우드 네이티브 환경으로 전환 및 신규 구축하는 프로젝트를 진행하려고 한다. TA(Technical Architect)와 AA(Application Architect)에 대하여 다음 내용을 설명하시오.
- 가. TA와 AA의 역할 비교 (범위, 책임/목표, 주요 산출물 측면)
- 나. TA와 AA 협업의 중요성 및 협업 방안
5. 소프트웨어 유지보수를 위한 재공학(Re-engineering)과 역공학(Reverse-engineering)에 대하여 다음 내용을 설명하시오.
- 가. 재공학 개념 및 목적
- 나. 재공학 절차
- 다. 역공학 개념 및 활용 방안
6. 프로그램 실행 시 운영체제는 메모리를 4가지 주요 영역으로 나누어 할당 및 관리한다. 각 영역의 역할과 특징 그리고 영역 간 동작 메커니즘에 대하여 다음 내용을 설명하시오.
- 가. 코드(Code)
- 나. 데이터(Data)
- 다. 힙(Heap)

01	오픈소스 DBMS		
문제	최근 상용 DBMS를 오픈소스 DBMS로 전환하는 수요가 늘고 있다. 다음 내용을 설명하시오. 가. 오픈소스 DBMS 전환 배경 나. 오픈소스 DBMS 전환 시 검토해야 할 제약사항 다. 오픈소스 DBMS 전환 시 단계별 마이그레이션 절차 라. 신뢰성 확보를 위한 고가용성(HA) 아키텍처 구성 방안		
도메인	디지털서비스	난이도	중 (상/중/하)
키워드	SPOF 제거, 자동 Failover, 데이터 복제, RPO/RTO 관리, 백업·모니터링		
출제배경	클라우드 전환으로 인해 오픈소스 DBMS 전환 가속화		
참고문헌	ITPE 서브노트		
출제자	남북평일야간반 전일 기술사(제 114회 정보관리기술사 /nikki6@hanmail.net)		

I. 오픈소스 DBMS 전환 배경

관점	전환 배경	주요 내용
경영·비용 관점	비용 효율성 확보	- 상용 DBMS 라이선스·유지보수 비용 증가 - 장기 운영 관점의 TCO 절감 요구
기술 관점	기술 성숙도 향상	- 오픈소스 DBMS 성능·안정성·기능 성숙 - 표준 SQL 기반 호환성·이식성 확보
아키텍처 관점	클라우드·분산 환경 대응	- 클라우드·MSA 확산 - 수평 확장·분산 구조 적용 용이
운영 관점	운영 유연성 확보	- 벤더 중심 정책·버전 종속 해소 - 커뮤니티·서브스크립션 기반 지원 확대
전략·정책 관점	기술 자립·정책 대응	- Vendor Lock-in 탈피 요구 - 공공부문 오픈소스 활성화 정책
미래 확장 관점	신기술 연계 및 확장성	- AI·빅데이터·실시간 분석 연계 용이 - 멀티·하이브리드 클라우드 대응

- 오픈소스 DBMS 전환은 벤더 종속과 라이선스 비용 부담을 해소하고, 기술 자율성과 확장성을 확보하여 디지털 전환 및 클라우드 환경에 유연하게 대응하기 위한 전략적 선택

II. 오픈소스 DBMS 전환 시 검토해야 할 제약사항

가. 오픈소스 DBMS 전환 시 기술·시스템 측면의 제약사항

관점	제약 요인	주요 제약사항
기능·기술	기능 격차	- 상용 DBMS 고유 기능 미지원 - 파티셔닝, 병렬처리, 고급 인덱스 기능 차이
SQL·개발	SQL·문법 비호환	- 벤더 종속 SQL, 힌트 미지원 - Stored Procedure·Trigger 재개발 필요
데이터	데이터 타입·정합성	- DATE, NUMBER, CLOB 타입 매핑 이슈 - 문자셋·Collation 차이

성능	성능 저하 가능성	- 대용량 트랜잭션 처리 성능 차이 - 복잡 쿼리·동시성 처리 한계
가용성	HA 구성 복잡성	- 이중화·자동 Failover 직접 구성 필요 - 복제 지연·Split-brain 위험
보안·규제	보안·감사 기능 차이	- 접근통제·감사 기능 제한 - 금융·공공 규제 충족 여부 검토 필요

나. 오픈소스 DBMS 전환 시 운영·조직·관리 측면의 제약사항

관점	제약 요인	주요 제약사항
운영	운영 체계 차이	- 백업·복구·모니터링 도구 성숙도 차이 - 운영 표준·자동화 체계 재정립 필요
장애 대응	장애 관리 미숙	- 장애 진단·복구 경험 부족 - Failover 시 운영 혼선 가능
인력·조직	기술 역량 전환	- DBA·개발자 재교육 필요 - 오픈소스 전문 인력 부족
지원 체계	기술 지원 한계	- 벤더 SLA 수준 지원 부재 가능성 - 커뮤니티 기반 지원의 불확실성
책임 구조	책임 소재 불명확	- 장애·보안 사고 시 책임 주체 모호
전환 리스크	서비스 연속성	- 전환 중 장애 발생 시 업무 영향 - 병행 운영 비용·복잡성 증가

- 오픈소스 DBMS 전환 시에는 기능·성능 차이, 애플리케이션 이식성, 안정성 검증, 기술 지원 체계, 전문 인력 확보, 보안 및 운영 관리 측면의 제약사항을 종합적으로 검토 필요

III. 오픈소스 DBMS 전환 시 단계별 마이그레이션 절차

가. 데이터베이스 마이그레이션 방식

방식	설명
Rehost	- Lift and shift, 동일 데이터베이스 엔진 및 코드 사용
Replatform	- 데이터베이스 엔진 변경 및 데이터 스키마 및 애플리케이션 코드 변경
Rebuild	- 작은 데이터 단위로 분할 최적화, 워크로드에 적합한 데이터로 변경

- 데이터베이스 마이그레이션 시나리오는 패턴에 따라 Rehost, Replatform, Rebuild 형태로 구분

나. 데이터베이스 마이그레이션 절차

절차	세부 절차	설명
준비	평가	- 소스 데이터 스키마 및 데이터 속성 분석 - 사이즈, 데이터 타입
	워크로드 분석	- 데이터베이스에 요구성능 및 종속성 분석 - 요구 처리 트래픽, 데이터 양, 애플리케이션 연동
계획	마이그레이션 계획 수립	- 마이그레이션 패턴 선택 및 세부 절차 설계

	테스트 계획	- 데이터 검증 테스트 계획 수립
마이그레이션	스키마 변환	- 이종 데이터베이스 전환 시 스키마 변환 수행 - 클라우드, 3rd party 도구 선택
	데이터 마이그레이션	- 원본 또는 변환 데이터 이동 - 온라인 또는 오프라인 선택
	애플리케이션 수정	- 변환 데이터베이스에 따른 쿼리 수정
	마이그레이션 테스트	- 기능, 성능테스트(응답시간, 쿼리 테스트)
	상용 전환	- 최종 데이터베이스 동기화 및 상용전환 결정
운영 및 최적화	모니터링	- 성능, 경고 모니터링 체계 구성
	백업	- 백업 체계 구성
	고가용성	- 장애대응 고가용성 최적화

- 원본 데이터베이스를 분석하고 데이터베이스 엔진 및 스키마를 검토, 이후 대상 데이터베이스를 설계하고 데이터베이스 엔진 또는 스키마를 변경할지 결정하는 단계로 진행

IV. 신뢰성 확보를 위한 고가용성(HA) 아키텍처 구성 방안

가. HA 구성 목표와 원칙

구분	목표	구성 원칙
① 가용성 확보	서비스 중단 최소화	- 모든 계층 SPOF 제거 - DB·네트워크·스토리지 이중화
② 신속한 장애 대응	장애 시 즉각 전환	- 자동 장애 감지 및 Failover - 수동 개입 최소화
③ 데이터 보호	데이터 유실 방지	- RPO 최소화 목표 설정 - 동기/비동기 복제 전략 수립
④ 복구 시간 단축	빠른 서비스 복구	- RTO 최소화 목표 설정 - 전환·복구 절차 자동화
⑤ 운영 안정성	장애 확산 방지	- Split-brain 방지(Quorum, fencing) - 장애 영역 분리(AZ/노드)

나. 신뢰성 확보를 위한 고가용성(HA) 아키텍처 구성 방안

구분	구성 요소	구성 방안
접속·분산 계층	접속 엔드포인트	- 단일 접속 지점(VIP/DNS) 제공으로 장애 시 접속 단순화
	로드밸런싱	- Read/Write 분리, Read Replica 트래픽 분산
DB·복제 계층	DB 이중화	- Primary-Standby 구조로 노드 간 이중화
	데이터 복제	- 동기/비동기 복제 병행으로 RPO 목표 충족
장애 대응 계층	장애 감지	- Heartbeat·Health Check 기반 상태 감시
	자동 전환	- Standby 자동 승격(Promote) 및 VIP 이동
데이터 보호 계층	백업	- Full + 증분/WAL(Log) 백업
	복구 검증	- 정기 복구 리허설로 백업 무결성 검증
운영·관제 계층	모니터링	- 성능·복제 지연·리소스 실시간 감시

	알림	- 임계치 기반 Alert 및 장애 조기 통보
--	----	---------------------------

- HA 아키텍처는 접속 분산, DB 이중화, 복제, 장애 자동전환, 백업·모니터링의 5개 핵심 요소로 구성하여 가용성과 데이터 신뢰성을 확보

“끝”

02	ISP(Information Strategy Planning)		
문제	다음 내용을 설명하시오. 가. ISP(Information Strategy Planning)의 정의 및 목적 나. ISP 수행방법론 체계와 절차 다. ISP, ISMP(Information System Master Plan) 비교		
도메인	IT경영전략	난이도	중 (상/중/하)
키워드	전략 정합성, 중·장기 비전, To-Be 아키텍처, 이행 로드맵		
출제배경	정보화 전략 수립과 시스템 구축 계획 간 차이를 명확히 이해하고, 단계별 정보화 기획 체계를 설명할 수 있는 역량을 검증하기 위한 출제		
참고문헌	ISP-ISMP 수립 공통가이드 제9판, ITPE 모의고사 및 서브노트		
출제자	정주행 조종흥 기술사(제127회 정보관리기술사 / jongheung.cho@gmail.com)		

I. ISP(Information Strategy Planning)의 정의와 목적

가. ISP(Information Strategy Plan) 정의

개념	- 조직 내의 전략적 정보 요구를 식별하고, 업무활동과 이에 대한 자료 영역을 기술하며, 정보시스템 개발을 위한 통합된 프레임워크를 제공하고, 이의 구현을 위한 통합 정보시스템 계획
필요성	- 경영 전략과 정보화 전략 간 정합성 확보 - 중·장기 관점의 정보화 비전 및 방향성 정립 - 정보시스템 중복·비효율 제거 및 투자 효율성 제고 - 미래 환경 변화에 대응 가능한 To-Be 정보화 구조 마련 - 단계적 정보화 추진을 위한 이행 로드맵 및 우선순위 수립

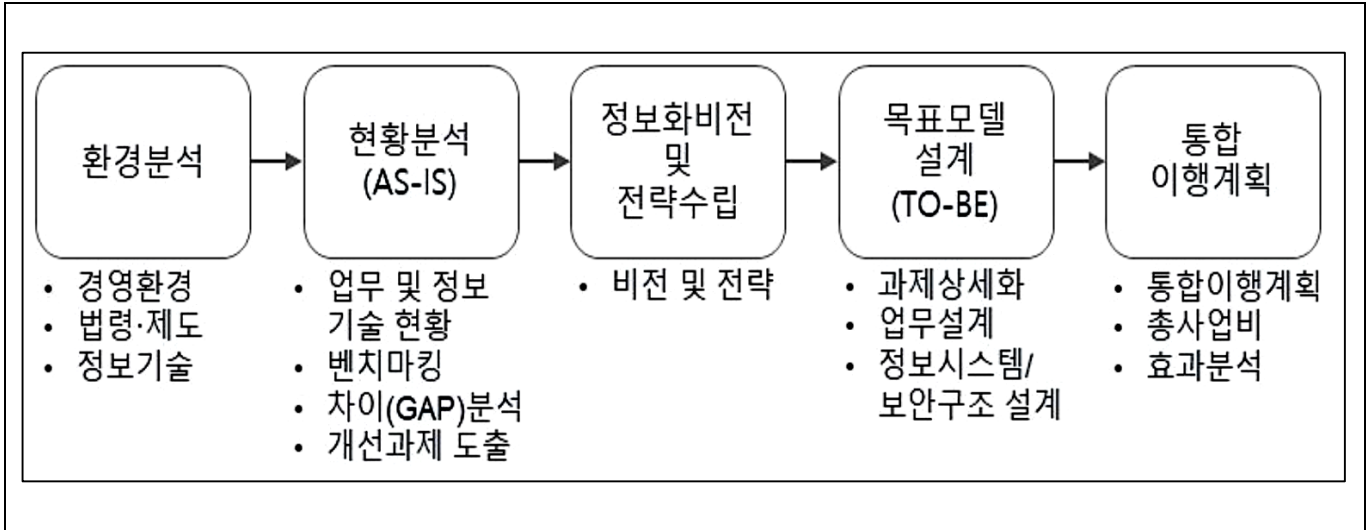
나. ISP(Information Strategy Plan) 목적

목적	설명
전략 정합성 확보	- 경영 전략과 정보화 전략 간 연계를 통한 전략적 정합성 확보
중장기 정보화 방향 수립	- 정책·기술·경영 환경 변화를 반영한 중·장기 정보화 비전 및 방향성 정립
As-Is 현황 진단	- 업무, 시스템, 데이터, 보안, 운영 전반에 대한 현황 분석 및 문제점 도출
To-Be 정보화 모델 정의	- 업무·데이터·애플리케이션·기술 아키텍처 관점의 To-Be 정보화 모델 정의
정보화 과제 도출	- 전략 목표 달성을 위한 실행 중심 정보화 과제 식별
우선순위 및 로드맵 수립	- 중요도·시급성·효과성 기반 정보화 과제 우선순위 설정 및 단계별 로드맵 수립
투자 효율성 제고	- 합리적인 IT 투자 기준 마련을 통한 비용 효율성 및 투자 타당성 확보

거버넌스 체계 정립	- 정보화 추진을 위한 의사결정·관리·통제 거버넌스 체계 확립
성과 관리 기반 마련	- 정보화 추진 성과의 측정·관리·개선을 위한 성과 관리 기준 수립

II. ISP 수행방법론 체계와 절차

가. ISP 수행방법론의 체계



- 환경·현황 분석을 통해 전략을 수립하고, To-Be 목표모델과 통합 이행계획으로 구체화하는 ISP 수행 체계

나. ISP 수행방법론의 절차

개념	- 조직 내의 전략적 정보 요구를 식별하고, 업무활동과 이에 대한 자료 영역을 기술하며, 정보시스템 개발을 위한 통합된 프레임워크를 제공하고, 이의 구현을 위한 통합 정보시스템 계획		
단계	활동	세부내용	산출물
환경분석	경영환경분석	- 외부환경 요인과 경영전략 분석을 통해 변화를 유발하는 요인에 대응하기 위한 시사점 도출	- 경영환경 분석서
	법령, 제도 분석	- 관련 법/제도 분석을 통해 사업에 영향을 미칠 수 있는 요구사항을 도출하여 목표모델 설계시 반영	- 법·제도 분석서
	정보기술(IT) 환경분석	- 최신 정보기술 추세와 기술환경 변화를 검토하여 최신 정보기술의 적용가능성 및 적용 사례 분석 - [국민중심] 정보기술 도입 분석 결과	- 정보기술 동향 분석서
현황분석 (As-Is 분석)	업무현황분석	- 업무절차맵(Process Modeling), 업무기능(Activity) 정의서 등을 작성하여 현행 조직과 업무체계상의 문제점 및 개선 요구사항을 도출	- 업무현황 분석서 (인터뷰 결과 포함)
	정보기술(IT) 현황분석	- (업무시스템 분석) 업무시스템 현황을 분석/진단하여 문제점 및 개선요구사항을 도출 - (데이터분석)데이터 현황 분석/진단하여 문제점 및 개선요구사항 도출 - (인프라분석) 하드웨어, 소프트웨어, 네트워크, 스토리지 등 현행 인프라를 분석/진단하여 문제점 및 개선요구사항 도출	- 정보기술 업무현황 분석서

		- (IT거버넌스 분석) IT업무(관리) 프로세스를 분석/진단하여 문제점 및 개선요구사항을 도출	
	벤치마킹	- 현황분석(업무&정보기술)을 통해 도출된 문제점 및 개선요구사항을 바탕으로 벤치마킹 대상(항목)을 선정 후, 선진사례 조사/분석을 진행	- 선진사례 동향 파악서
	차이(Gap)분석	- 선진사례의 업무절차 및 정보기술 요건을 도출한 후, 기도출된 정보화 요건과의 차이를 분석, 과제의 보완작업 및 개선	- 차이분석서
	이슈통합 및 개선과제 도출	- 도출된 이슈를 종합, 연관성 높은 이슈사항 그룹화 - 근본원인 분석 후 개선과제 도출 - [국민중심] 사용자 편의성 향상 과제 도출 결과	- 요구사항 및 개선과제 분석서
정보화비전 및 전략 수립		- 환경분석/현황분석 결과를 연계하여 비전, 목표, 단계별 실행전략 등을 수립 - [민관협력] 민관협력 대상,분야,방식 등 협력 전략 수립 - [클라우드 기술 우선 적용 권고] 민간 클라우드 및 디지털서비스 전문 계약제도 이용 방향	- 정보화 전략 정의서
목표모델 설계 (To-Be Model)	To-Be 개선 과제 상세화	- 현황분석시 도출된 개선과제의 상세화(과제 개요, 추진 범위, To-Be 개선방향, 적용사례 등) - [AI·데이터기반] AI·데이터 기술기반 의사결정 활용 계획	- To-Be 과제 상세 정의서
	To-Be 업무 프로세스 설계	- 개선과제 내역, 선진사례, IT 개선방향을 종합적으로 고려하여 최적화된 To-Be 업무프로세스 재설계 - To-Be 업무프로세스 내 요건 상세 정의	- To-Be 업무프로세스 설계서
	To-Be 정보 시스템 구조 설계	- 전략적 정보시스템 구축을 위한 이상적 응용서비스 구조를 정립 - [국민중심] 통합인증체계 도입 계획 - [국민중심] 사용자 이용환경을 고려한 설계	- To-Be 정보시스템 구조 설계서
	To-Be 데이터 구조 설계	- 정립된 정보시스템을 효율적으로 운용할 정보자원(데이터) 관리 체계를 정리 - [하나의정부] 범정부 시스템연계,개방형 표준 적용 계획	- To-Be 데이터 구조 설계서
	To-Be 기술 및 보안 구조설계	- 전략적 정보시스템 구축을 위한 필요 기술 요소 및 기반(인프라) 구조를 정립	- To-Be 기술 및 보안 구조 설계서
통합 이행 계획	통합 이행계획 수립	- 과제별 우선순위 평가 및 전략적 특성, 연관성 바탕으로 추진 체계 및 시행일정 수립	- 통합 이행계획 수립서
	총구축비 산출	- (SW 개발비) 기능점수(FP)산정 후, 개발비 도출 - (장비비) SW구매, HW구매 등 항목별 규격, 수량, 금액 내역	
	효과 분석	- 타당한 기대효과 분석	

- 환경분석 → 현황분석 → 전략수립 → 목표모델 설계 → 통합 이행계획 수립의 단계적 절차 구성

III. ISP, ISMP(Information System Master Plan) 비교

가. ISP, ISMP 개념 측면의 비교

구분	ISP	ISMP
개념	- 조직의 경영 전략을 지원하기 위한 중·장기 정보화 전략 수립 활동	- 특정 정보시스템 구축을 위한 실행 중심의 마스터플랜 수립 활동
지향점	전략 중심(Why, What)	실행 중심(How)
역할	정보화 방향성 및 투자 기준 제시	정보시스템 구축을 위한 요구사항과 실행 계획 구체화
시간적 관점	중·장기 관점	단기·중기 관점
적용 시점	정보화 추진의 출발 단계	사업 착수 전 또는 ISP 이후 단계

나. ISP, ISMP 상세 비교

구분	ISP	ISMP
목적	- 경영 전략과 정보화 전략의 연계를 통한 중·장기 정보화 방향성 정립	- 특정 정보시스템의 기능적·기술적·비기능적 요구사항 상세화
범위	- 전사, 서비스 영역 또는 부서 단위 정보화 전략	- 단위 프로젝트 또는 프로젝트 묶음
설계 대상	- 조직의 경영 목표 및 전략 기반 정보화 체계	- SW 사업 범위 내 업무 및 정보시스템
상세화 수준	- 이행 과제 수준까지 정의하되, 세부 구현은 RFP 단계에서 구체화	- 기능 점수 산정이 가능한 수준까지 요구사항 상세 기술
주요 관점	- 경영, 업무, 정보기술, 데이터, 보안, 거버넌스	- 기능, 데이터, 트랜잭션, 성능, 테스트, 운영
주요 활동	- 경영환경 및 정보기술 동향 분석 - 업무 및 정보시스템 현황 분석 - 정보시스템 구조 분석 및 정보관리체계 수립 - 미래 업무 프로세스 및 정보시스템 구조 설계 - To-Be 로드맵 수립	- 정보시스템 구축 범위 및 방향 수립 - 기능·기술·비기능 요구사항 도출 - 정보시스템 구조 및 요구사항 상세 기술 - 구축 계획 수립 - 예산 산정 및 사업자 선정·평가 지원
주요 산출물	- 환경분석 및 정보기술 동향 분석 보고서 - 업무·정보시스템 현황 분석 보고서 - IT 비전 및 정보화 전략 - 이행 과제 및 중·장기 로드맵	- RFP(제안요청서) - 정보시스템 예산 산정 자료 - 요구사항 정의서

- ISP는 정보화 전략과 방향성을 제시하는 상위 계획이며, ISMP는 해당 전략을 구체적인 시스템 구축 수준으로 전환하는 실행 계획임

IV. ISP·ISMP 수립 공통가이드(제 9 판) 주요 개정 사항

항목	현행(제 8 판)	개정(제 9 판)	
신규제도	- 없음	- 소규모 정보시스템 구축 사업 ISP-ISMP 수립 의무 면제 (사업계획 수립, 검토 진행)	- 소규모 정보시스템구축 사업기간 단축 및 자원투입 절감
대상 및 비용	- 없음	- 각 중앙관서에서 추진하는 총 구축비 20 억원 미만의 정보시스템 구축, 재구축사업	
주요 절차	- 없음	① 사업계획서 수립(보완) 및 검토요청 ② 사업계획서 사전검토 ③ 사업계획서 충실성 판단 ④ 검토의견서 작성 및 기획재정부 제출 ⑤ 예산편성시 검토의견서 참작	

“끝”

03	GPU와 TPU		
문제	인공지능 학습에 이용하는 GPU(Graphics Processing Unit)와 TPU(Tensor Processing Unit)에 대하여 다음 내용을 설명하시오. 가. GPU와 TPU 개념 나. GPU와 TPU 비교 다. 최근 GPU보다 TPU를 사용하는 이유 라. 효율성 측면에서 TPU의 장점 및 향후 전망		
도메인	CA	난이도	중 (상/중/하)
키워드	병렬처리 아키텍처(Parallel Architecture), 행렬연산 가속(Matrix Computation Acceleration) Systolic Array , 데이터이동 최소화(Data Movement Optimization), 에너지 효율성(Performance per Watt)		
출제배경	대규모 AI 학습 환경에서 GPU의 구조적 한계를 인식하고, 연산·메모리·에너지 효율을 중심으로 TPU와 같은 목적형 가속기의 등장 배경과 활용 타당성을 이해하는지를 평가		
참고문헌	ITPE 모의고사, 서브노트		
출제자	정주행 조종흥 기술사(제127회 정보관리기술사 / jongheung.cho@gmail.com)		

I. GPU 와 TPU 의 개념과 특징

가. GPU 와 TPU 의 개념

GPU	TPU
그래픽 처리를 위해 개발됐으나, 수천 개의 코어로 병렬 연산 을 수행해 AI와 범용 계산에 널리 쓰이는 고성능 프로세서	구글이 딥러닝의 핵심인 행렬 연산을 가속화하기 위해 독자 개발한 AI 전용 주문형 반도체(ASIC) 로, 학습과 추론 효율을 극대화한 장치.

나. GPU 와 TPU 의 특징 비교

GPU		TPU	
범용성	그래픽 AI, 과학연산등의 범용적 활용	전문성	딥러닝의 대규모 행렬 연산에만 특화
병렬성	수천 개의 코어로 데이터를 동시에 처리	효율성	불필요한 제어를 줄여 전력과 공간 절약
접근성	누구나 구매하여 다양한 환경에 구축가능	최적화	텐서플로우 환경에서 처리 속도가 높음

II. GPU 와 TPU 의 개념도 비교 및 상세 비교

가. GPU 와 TPU 의 개념도 비교

구분	GPU	TPU
----	-----	-----

개념도		
주요특징	- 중앙의 처리와 각 셀의 병렬 처리	- 중앙 없이 모든 셀이 연결되는 구조

나. GPU 와 TPU 의 상세 비교

구분	GPU	TPU
핵심구조	- SIMT(Single Instruction, Multiple Threads)	- Systolic Array(시스톨릭 배열)
작동원리	- 수천개의 코어가 병렬 작동하되, 코어그룹은 독립적인 코어를 가짐	- 수만개의 연산 유닛이 격자형태로 연결 - 각 유닛이 서로서로 얹혀 있음
주된 용도	- 그래픽 처리(Gaming, 3D 렌더링) 범용 AI학습 및 추론, 과학 연산	- 초대형 딥러닝 모델의 학습 및 추론 - 대규모 언어모델(LLM)의 학습 및 추론
강점	- (유연성) 다양한 병렬 연산 가능 - (생태계) CUDA의 개발환경	- (효율성) 행렬연산의 압도적 정렬 효율 - (최적화) 메모리 병목 현상 적음
약점	- 전력소모의 높음 - 연산코어 비율이 상대적 낮음	- 딥러닝 이외의 작업 불가 - 유연성이 떨어짐
프레임워크	- Pytorch, Tensorflow - 모든 AI 프레임워크	- Tensorflow, JAX 가장 최적화 - Pytorch 향후 지원
GPU연결	- NVlink & NVSwitch	- ICI(Inter-Chip Interconnect)
GPU연결구조	- Switch 기반 (Switch-based) - 중앙 NVSwitch 를 GPU 1:1 초고속 통신	- 토러스 기반 (Torus Topology) - 칩끼리 상하좌우(3D)직접 연결

- Network ACL 은 큰 서버넷을 방어하는 구간에서 Security Group 은 개별 VPC 를 방어하는 구간에서 사용

III. 최근 GPU 보다 TPU 를 사용하는 이유

가. 연산 아키텍처 관점의 TPU 를 사용하는 이유

구분	GPU 한계	TPU 강점
설계 철학	- 그래픽·과학연산·AI를 모두 고려한 범용 병렬 프로세서	- 딥러닝 학습·추론만을 목표로 한 목적형 프로세서
연산 방식	- SIMT 기반으로 제어 흐름·스케줄링 오버헤드 존재	- Systolic Array 기반 행렬 연산 으로 제어 오버헤드 최소화
AI 연산 효율	- 다양한 연산을 처리하나, 행렬 연산에 최적화 한계	- 행렬 곱(MAC) 연산을 하드웨어 수준에서 극대화
파이프라인 활용	- 스레드 간 동기화·분기 처리로 효율 저하 가능	- 연산이 파이프라인처럼 연속적으로 흐름

불필요 회로	- 캐시 제어, 분기 처리, 스케줄러 등 AI에 불필요한 로직 포함	- AI 연산에 불필요한 로직 제거
--------	---------------------------------------	---------------------

나. 메모리 및 데이터 이동 관점의 TPU 를 사용하는 이유

구분	GPU	TPU
병목 요인	- 연산 성능 대비 메모리 접근 지연이 병목	- 데이터 이동을 최소화하도록 구조적으로 설계
메모리 구조	- HBM-L2-L1-레지스터 계층 구조로 데이터 이동 빈번	- 대용량 On-chip 메모리 중심 구조
데이터 재사용	- 캐시 의존 → 반복 연산 시 재적재 발생	- 가중치·활성값을 칩 내부에서 반복 재사용
전력 효율	- 데이터 이동이 많아 전력 소모 증가	- 데이터 이동 감소로 전력 효율 극대화
대규모 모델	- 모델 규모 증가 시 메모리 병목 심화	- 대규모 딥러닝 모델에 안정적 확장성 제공

IV. 효율성 측면의 TPU 의 장점 및 향후 전망

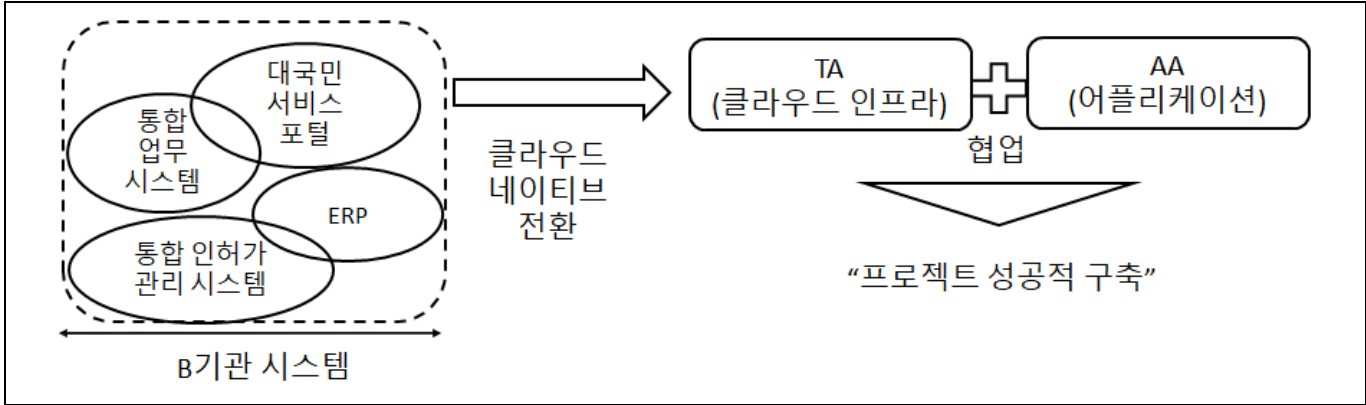
구분	TPU의 장점(효율성 중심)	향후 전망
연산 효율	- 행렬 곱(MAC) 연산을 Systolic Array 로 하드웨어 구현하여 연산 밀도 극대화	- 대규모 언어모델(LLM)·멀티모달 모델 학습에 특화된 초대형 행렬 연산 가속기 로 진화
에너지 효율	- 불필요한 제어·캐시·스케줄링 로직 제거로 연산 대비 전력 소모 최소화	- 데이터센터 전력·탄소 절감 요구에 따라 친환경 AI 가속기 표준으로 확산
메모리 효율	- On-chip 메모리 중심 설계로 데이터 이동 비용 최소화	- 메모리 병목을 구조적으로 해결하는 Dataflow 기반 아키텍처 가 AI 가속기의 기본 모델로 정착
처리량 (Throughput)	- 동일 전력 대비 GPU보다 높은 연산 처리량(TOPS/W) 제공	- 추론 전용 TPU, 학습 전용 TPU 등 워크로드 분화형 TPU 등장
확장 효율	- 다수의 TPU를 묶어도 통신 오버헤드가 낮은 스케일 아웃 구조	- AI 전용 인터커넥트 기반의 초대규모 AI 클러스터 로 발전
비용 효율	- 동일 성능 대비 전력·냉각·운영 비용 절감	- AI 인프라 TCO 절감을 위한 클라우드 기반 TPU 사용 증가
소프트웨어 효율	- TensorFlow, XLA 기반으로 컴파일 단계에서 연산 최적화 자동화	- 모델-컴파일러-하드웨어가 결합된 HW-SW Co-design 가속 심화

- TPU 는 연산·에너지·메모리 효율을 구조적으로 극대화한 AI 전용 가속기로, 향후 대규모 AI 모델 확산과 친환경 데이터센터 요구에 따라 GPU 중심 구조를 보완·대체하는 핵심 인프라로 발전할 것으로 보임.

“끝”

04	TA(Technical Architect), AA(Application Architect)		
문제	B기관은 기존에 운영 중이던 대국민 서비스 포털, 통합 업무 시스템, ERP 및 통합 인허가 관리 시스템을 클라우드 네이티브 환경으로 전환 및 신규 구축하는 프로젝트를 진행하려고 한다. TA(Technical Architect)와 AA(Application Architect)에 대하여 다음 내용을 설명하시오. 가. TA와 AA의 역할 비교 (범위, 책임/목표, 주요 산출물 측면) 나. TA와 AA 협업의 중요성 및 협업 방안		
도메인	소프트웨어공학	난이도	중 (상/중/하)
키워드	컨테이너, 마이크로서비스, Ci/CD, Auto Scaling, 인프라, 어플리케이션		
출제배경	클라우드 전환 시 AA와 TA의 역할 이해		
참고문헌	ITPE 기술사회 자료		
출제자	NS반 백현 기술사(제 122회 정보관리기술사 / snuoo@naver.com)		

I. B 기관의 클라우드 네이티브 환경으로의 시스템 전환 개요



- B 기관에서 운영중인 시스템을 클라우드 네이티브 환경으로 전환 시 클라우드 기반의 일관된 프레임워크 구축 및 운영을 위해 TA와 AA의 협업이 필요하고, 성공적인 프로젝트 완료 수행이 가능

II. TA(Technical Architect)와 AA(Application Architect)의 역할 비교

가. 범위 측면의 역할 비교

비교	TA(Technical Architect)	AA(Application Architect)
인프라 자원 설계	- CSP(AWS, Azure 등) 서비스 선정, 리전/ 가용영역(AZ) 설계	- 앱 구동을 위한 런타임 환경, 라이브러리 및 종속성 관리
플랫폼 소프트웨어	- K8s(Kubernetes), WAS, DB, WEB 서버 설 치 및 튜닝	- 공통 프레임워크 설계, API 게이트웨이 및 메 시지 큐 연동
연결성 및 보안	- VPC, 서브넷, 방화벽, 로드밸런싱, SSL 인 증 설계	- 서비스 간 인터페이스 규격, 인증/인가 연동 로직
운영 체제 및 런타임	- OS 커널 최적화, 컨테이너 런타임 (Docker/CRI-O) 설정	- Java/Python 등 언어 버전, Framework(Spring Boot 등) 선정

나. 책임/목표 측면의 역할 비교

비교	TA(Technical Architect)	AA(Application Architect)
시스템 성능	- 고가용성(HA), 부하분산(Scalability), 백업/복구 체계 확보	- 응답 속도 최적화, 모듈 간 낮은 결합도, 높은 재사용성
품질 및 표준	- 시스템 성능 벤치마킹(BMT), 리소스 가용량 산정(Sizing)	- 소스 코드 표준 가이드 준수, 테스트 자동화 체계 수립
운영 효율성	- 시스템 자동 관제, 로그 수집, 인프라 비용(FinOps) 최적화	- CI/CD 배포 파이프라인 최적화, 오류 추적 및 디버깅 용이성
준거성	- ISMS-P 등 정보보호 체계에 따른 인프라 보안 통제	- 개인정보 암호화, 웹 취약점(OWASP) 방지 로직 적용

다. 주요 산출물 측면의 역할 비교

비교	TA(Technical Architect)	AA(Application Architect)
아키텍처 구성도	- 시스템 아키텍처 구성도, 하드웨어/SW 배치도	- 애플리케이션 아키텍처 정의서, 컴포넌트 구성도
설계 및 가이드	- 네트워크 설계서, 보안 설계서, 클라우드 자원 목록	- 개발 표준 가이드서, 공통 컴포넌트 설계서, 인터페이스 정의서
테스트 결과서	- 부하 테스트 결과 보고서, 가용성 검증 결과서	- 단위/통합 테스트 시나리오, 소스 정적 분석 결과서

- TA는 클라우드 인프라, AA는 클라우드 인프라 기반에서 실행되는 표준 프레임워크 제공이 핵심

III. TA와 AA 협업의 중요성 및 협업 방안

가. TA와 AA 협업의 중요성

구분	협업의 중요성	설명
구축 측면	MSA 서비스 간 최적화	- 대국민 포털부터 ERP까지 서비스를 MSA환경에서 구축 시 AA는 서비스 호출 설계, TA는 서비스 메시 및 오케스트레이션 설정이 일치해야 지연 없는 서비스 가능
	컨테이너 기반의 환경 일관성 확보	- AA가 정의한 애플리케이션 실행 환경이 TA가 관리하는 컨테이너 이미지(Docker) 및 오케스트레이션(Kubernetes) 환경과 완벽히 동기화되어야 '개발-검증-운영' 환경차이의 오류를 원천 차단 가능
	무중단 배포 체계 (CI/CD)	- 시스템의 업데이트 시, AA의 배포 전략과 TA가 구축한 자동화 파이프라인이 긴밀히 연동되어야 무중단 서비스 제공 가능
	표준 프레임워크와 연계	- 로깅, 보안, 인증, 데이터베이스 연계 등 TA제공하는 서비스가 AA가 설계한 표준 프레임워크로 내제화 하여 일관된 환경 제공
운영 측면	클라우드 자원 Auto-scaling	- 대국민 포털에 접속자가 몰릴 때 AA가 설계한 Stateless 구조와 TA가 설정한 Auto-scaling 정책 연계로 탄력적 서비스 제공 가능
	고가용성 보장	- 복잡한 트랜잭션을 처리하기 위해 AA의 분산 트랜잭션 설계와 TA의 클라우드 DB 백업 전략이 통합되어 장애 시 데이터 유실 방지
	FinOps 협업	- AA가 애플리케이션의 리소스 사용 효율을 높이고, TA가 이에

		최적화된 클라우드 인스턴스 배치로 운영 예산 절감
	통합 모니터링 및 장애 대응 (Observability)	- AA의 애플리케이션 로그와 TA의 인프라 메트릭이 하나의 대시보드에서 관리되어 장애 발생 시 복구시간 최소화 가능

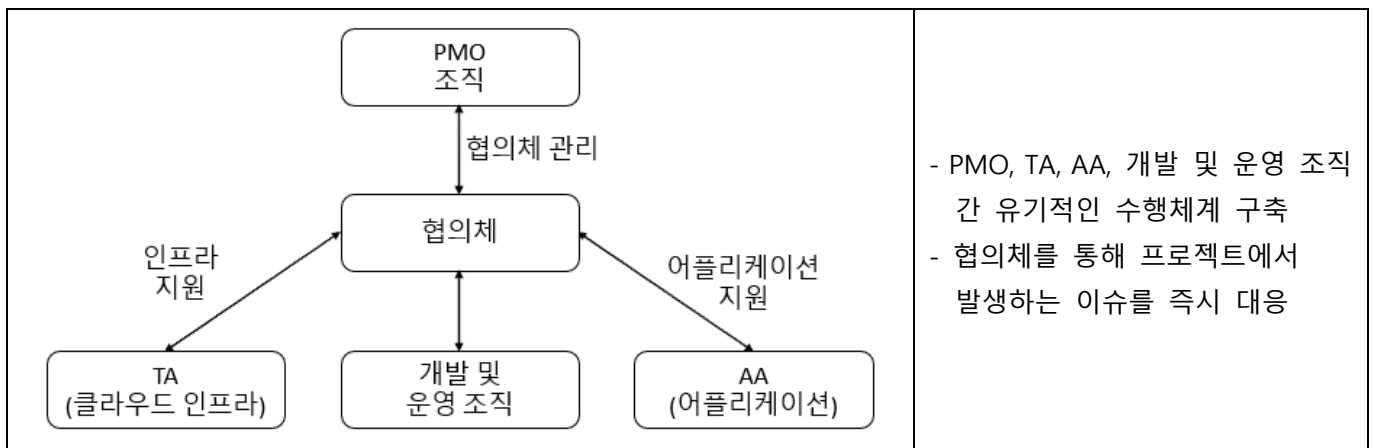
- 레거시 시스템이 클라우드 네이티브 환경에서 전환 및 신규 구축 시 TA와 AA의 역할이 핵심

나. TA와 AA 협업 방안

구분	협업 방안	설명
구축 측면	프로젝트 협업 위원회 신설	- TA의 인프라 설계와 AA의 애플리케이션 설계가 충돌하지 않도록 관리자가 주관하는 정기 리뷰 회의를 운영
	플랫폼 엔지니어링 기반 구축	- TA가 표준화된 개발 환경을 카탈로그화하여 제공하고, AA는 이를 개발 표준으로 채택하도록 관리
	공동으로 진행하는 검증 체계 구축	- AA의 어플리케이션 검증 시나리오와 TA의 인프라 성능검증 시나리오 등, 상호작용하여 검증이 진행되도록 체계 구축 및 성과관리 수행
	단일 CI/CD 파이프라인 내에서 통합 관리	- TA영역의 클라우드 인프라와 AA영역의 어플리케이션이 하나의 CI/CD 파이프라인 내에서 수행하도록 규정하여 관리
운영 측면	통합 모니터링 체계	- TA의 시스템 지표와 AA의 어플리케이션 로그를 하나의 통합 대시보드로 관리하여 장애 발생 시 유기적으로 대응하도록 체계구축
	무중단 배포 및 장애복구 훈련 관리	- AA의 무중단 배포 전략과 TA의 인프라 복구 시나리오를 결합한 모의훈련을 정례화하고 Pain Point를 지속적으로 개선
	FinOps 기반의 최적화	- 클라우드 비용 청구서 바탕으로 TA와 AA가 함께 불필요한 자원이나 비효율적인 로직을 찾아서 품질을 개선하는 회의 정례회
	협업 문화 정례화	- 비즈니스 이벤트에 대해서 TA와 AA가 공동으로 협업하여 원인을 분석하고 재발방지 대책을 수립하는 프로세스 의무화

- TA와 AA 협업을 위한 PMO 조직 기반 관리 체계 구축 필요

IV. B기관에서 TA와 AA 관리를 위한 수행체계



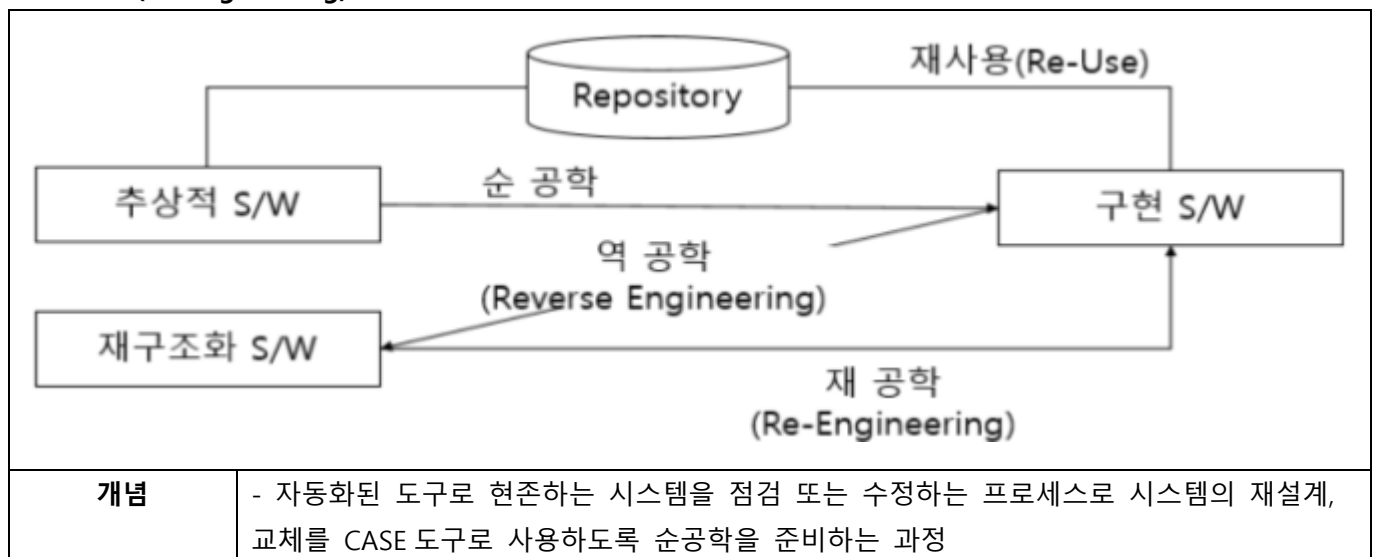
- 주기적인 협업체를 주관하여 B기관의 클라우드 네이티브 전환 및 신규 구축 시 의사소통 체계를 일원화하여 성공적인 프로젝트 종료가 가능

“끝”

05	소프트웨어 유지보수		
문제	소프트웨어 유지보수를 위한 재공학(Re-engineering)과 역공학(Remove-engineering)에 대하여 다음 내용을 설명하시오. 가. 재공학 개념 및 목적 나. 재공학 절차 다. 역공학 개념 및 활용 방안		
도메인	소프트웨어 공학	난이도	하 (상/중/하)
키워드	유지보수, 3R, de-compile, 문서화, 재구조화, 재사용		
출제배경	소프트웨어 유지보수의 기본내용 확인		
참고문헌	ITPE 기술사회 자료집		
출제자	정상반 정상 기술사(제 124회 정보관리기술사 / itpe_peak@naver.com)		

I. 소프트웨어 유지보수, 재공학(Re-engineering)의 개념 및 목적

가. 재공학(Re-engineering)의 개념



- 기존 운영시스템이 변화에 대해 유연성을 갖도록 하는 것이 목적

나. 재공학(Re-engineering)의 목적

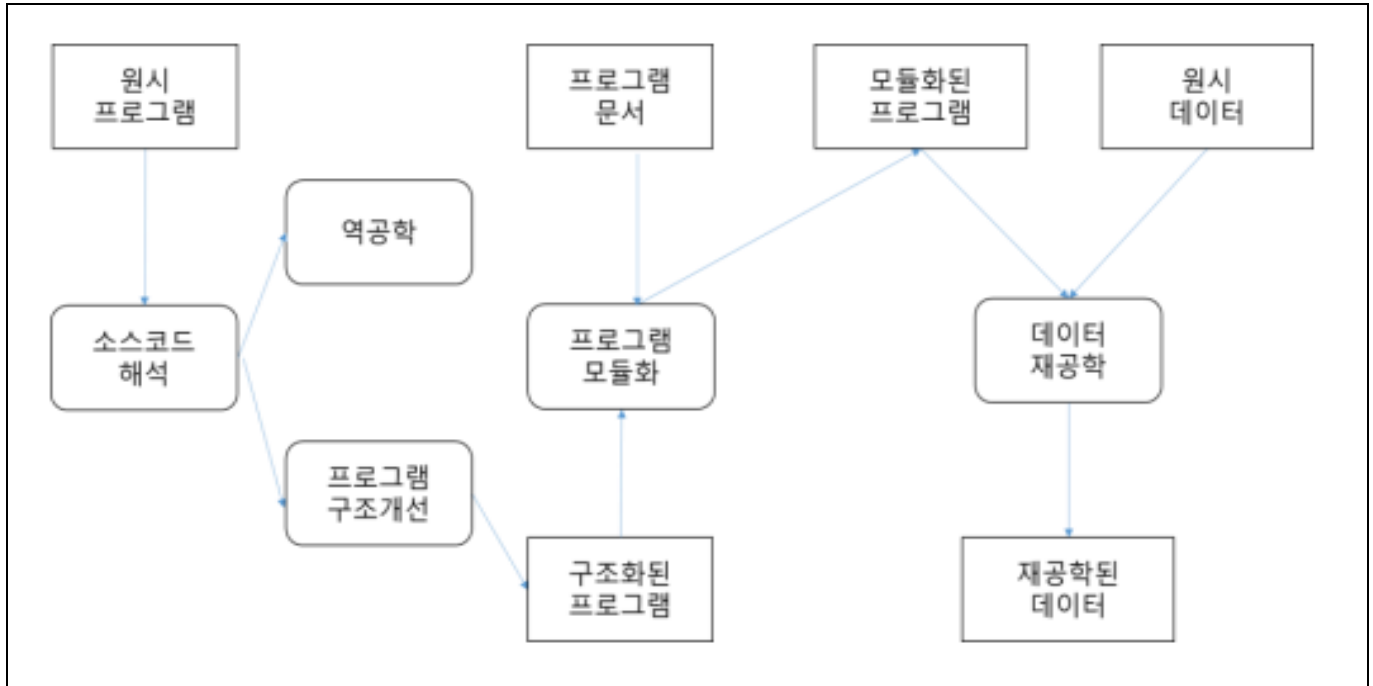
구분	목적	설명
유지보수성 향상	유지보수 비용 절감	- 기존 시스템의 복잡한 구조를 개선하여 향후 발생할 수정 및 보수 작업을 더 쉽고 저렴하게 만들
시스템 이해도 증진	가독성 및 분석 용이성 확보	- 문서화가 제대로 되어 있지 않거나 스파게티 코드로 얽힌 시스템을 분석하여 개발자가 구조를 명확히 파악
신뢰성 및 품질 개선	오류 감소 및 안정성 강화	- 노후화된 시스템 내부의 잠재적인 결함을 제거하고, 최신 표준을 적용하여 시스템의 전반적인 신뢰도를 높임
수명 연장 및 현대화	새로운 기술 환경 적응	- 기존의 비즈니스 로직은 유지하면서, 최신 하드웨어나 운영체제, 프레임워크에 맞게 시스템을 변환하여 가용성을 높임.

재사용성 확보	모듈화 및 부품화	- 특정 기능을 독립된 모듈로 분리하여 추출함으로써, 다른 시스템이나 프로젝트에서도 해당 기능을 다시 사용할 수 있도록 함
------------	-----------	--

- 재공학은 기존 시스템의 비즈니스 로직을 버리지 않으면서 낡은 부분만 새 것으로 교체한다는 것이 핵심

II. 재공학(Re-engineering)의 절차 설명

가. 재공학(Re-engineering)의 절차



- 신규 시스템이 이전 시스템보다 비용, 품질, 유지보수 측면 등의 개선 달성 필요함

나. 재공학(Re-engineering)의 절차 설명

절차	설명
① Reverse Engineering	- Source Code, 설계 문서 추출 - Target System 실행 file 의 구조(PE)등을 분석하고 Assembly Code 를 추출하는 단계
② 재구조화	- Target System 에서 추출한 code 와 repository 에서 획득한 code & 문서를 기반으로 의미론적 정보 추출과 Program 구조와 데이터를 재구조화 하는 단계
③ 구현	- 재구조화된 Program 과 Data 를 합성하여 성능이 개선된 Target System 을 Build 하는 단계

- LLM 기반 앱은 데이터 및 사용자 입력 처리 방식의 독특한 특성으로 인해 기존 IT 시스템과 다른 보안 위협에 취약해 철저한 대비가 필요

III. 역공학(Reverse Engineering)의 개념 및 활용 방안

가. 역공학(Reverse Engineering)의 개념

정의	- SW에 대한 디버깅, 디컴파일 등의 분석을 통해 기존 제품에 구현된 구조, 원리, 기술, 알고리즘 등을 역으로 분석하여 재구성하는 과정	
프로세스		
	① Code 추출	- Ollydbg, JD-GUI 등의 Tool 사용 - Dirty Code 추출
	② Code 분석 / 수정	- 정적/동적 분석 및 Clean Code 화 - WireShark 와 같은 툴을 사용하여 실제 동작 시 발생하는 Packet, Log file을 추출하여 분석
	③ 문서화	- 분석된 Source code와 해당 분야의 도메인 지식을 활용하여 Program, Data 구조에 대한 명세서 작성
종류	논리 역공학	- 원시 코드로부터 정보를 추출하여 물리적 설계 정보 저장소에 저장, 물리적 설계정보를 얻어 내는 수단
	자료 역공학	- 기존 데이터베이스를 수정하거나 새로운 데이터베이스 관리 시스템으로 전이하는 역할

- 기존 운영시스템이 변화에 대해 유연성을 갖도록 하는 것이 목적

나. 역공학(Reverse Engineering)의 활용 방안

구분	활용 방안	설명
시스템 분석 및 문서화	노후 시스템 설계 복원	- 원본 설계도가 사라진 오래된 프로그램의 실행 파일이나 소스를 분석하여, 시스템의 전체적인 구조도와 흐름도를 다시 생성
	데이터 구조 파악	- 데이터베이스 내 테이블 간의 관계(ERD)가 불분명할 때, 소스 코드의 쿼리문을 역추적하여 데이터가 흐르는 규칙을 명확히 정의
보안 점검 및 대응	악성코드 바이너리 분석	- 바이러스나 랜섬웨어를 가상 환경에서 실행해 보며 어떤 파일을 수정하고 어디로 데이터를 전송하는지 파악하여

		치료 방법 확인
	소프트웨어 취약점 탐지	- 해커가 이용할 수 있는 보안 구멍(오버플로우 등)을 찾기 위해 프로그램을 분해하여 내부 로직의 허점을 선제적으로 진단
유지보수 및 개선	코드 구조 정돈	- 복잡하게 꼬인 '스파게티 코드'의 호출 관계를 분석하여, 기능별로 모듈화하고 가독성 높은 코드로 재구성하는 기초 자료를 제공
	숨은 버그 로직 추적	- 일반적인 테스트로 발견되지 않는 논리적 오류를 찾기 위해, 실행 중 메모리 변화와 CPU 명령어를 직접 관찰하여 오류 발생 지점을 특정
상호 운용성 확보	하드웨어 호환 드라이버 개발	- 제조사가 드라이버를 제공하지 않는 장치의 통신 신호를 분석하여, 리눅스 등 다른 OS에서도 해당 장치가 작동하도록 소프트웨어를 만듦
	비공개 프로토콜 분석	- 서로 다른 회사 제품 간에 데이터를 주고받을 수 있도록, 비공개된 통신 규격(Message Format)을 해독하여 연동 모듈을 구현
지식 재산권 보호	특허 기술 도용 확인	- 자사의 독자적인 알고리즘이 경쟁사 제품에 몰래 사용되었는지 확인하기 위해, 상대방의 프로그램을 기계어 수준에서 분석하고 대조
	라이선스 위반 검증	- 유료 소프트웨어 내에 고지 없이 오픈소스가 사용되었는지, 혹은 라이선스 범위를 초과하여 복제되었는지 법적 근거를 수집하기 위해 활용

- 기존 시스템의 추상화된 설계 정보를 추출하여 재사용하거나 개선을 위해 역공학 적용

“끝”

06	메모리 주요영역		
문제	프로그램 실행 시 운영체제는 메모리를 4가지 주요 영역으로 나누어 할당 및 관리한다. 각 영역의 역할과 특징 그리고 영역 간 동작 메커니즘에 대하여 다음 내용을 설명하시오. 가. 코드(Code) 나. 데이터(Data) 다. 힙(Heap) 라. 스택(Stack)		
도메인	CA/OS	난이도	하 (상/중/하)
키워드	유지보수, 3R, de-compile, 문서화, 재구조화, 재사용		
출제배경	컴퓨터 구조 메모리의 기본 주요 영역 이해 확인		
참고문헌	ITPE 기술사회 자료집		
출제자	정상반 정상 기술사(제 124회 정보관리기술사 / itpe_peak@naver.com)		

I. 운영체제 메모리의 주요영역, 코드(Code), 데이터(Data) 설명

가. 코드(Code)의 설명

개념	- 실행할 프로그램의 기계어 명령(Instruction)들이 저장되는 공간	
역할	기계어 명령 저장	- CPU 가 직접 실행할 수 있는 바이너리 형태의 프로그램 명령어를 보관
	제어 흐름 관리	- 함수 호출, 분기문, 반복문 등 프로그램의 실행 로직(Text)이 위치
	라이브러리 연결	- 정적 링크된 라이브러리 함수들이 이 영역에 포함되어 실행 준비 완료
	엔트리 포인트 제공	- 프로그램이 시작되는 지점(main 함수 등)의 주소값을 보유
특징	Read-Only	- 실행 중 코드 변조를 통한 해킹(예: 코드 인젝션)을 방지하기 위한 읽기 전용 설정
	Fixed Size	- 컴파일 타임에 크기가 결정되어 프로그램 종료 시까지 크기 변화 없음
	Code Sharing	- 동일 프로그램을 여러 개 실행해도 메모리에는 하나만 올려 공유 (메모리 절감)
	하위 주소 배치	- 주소 공간의 최하단에 위치하여 상위 영역(Heap, Stack)의 간섭 최소화

- 소스 코드가 컴파일러에 의해 기계어로 번역된 후, 로더(Loader)에 의해 메모리에 적재된 상태

나. 데이터(Data)의 목적

개념	- 프로그램 시작과 동시에 할당되어 종료 시까지 유지되는 전역 변수와 정적(static) 변수의 저장소	
역할	전역 자원 관리	- 프로그램 어디서든 접근 가능한 전역 변수(Global Variable) 저장
	정적 데이터 유지	- static 변수를 저장하여 함수가 종료되어도 값을 소멸시키지 않고 보존

	상수 데이터 관리	- 선언된 상수(Const) 중 일부 또는 리터럴 데이터의 참조점 제공
	실행 환경 초기화	- 프로그램 시작 시 필요한 초기 설정값(Initial Values)을 메모리에 로드
특징	GVAR / BSS 분리	- 초기값 유무에 따라 Data(초기화 O)와 BSS(초기화 X)로 분리하여 바이너리 크기 최적화
	상주성(Persistence)	- 프로그램 시작 시 할당되고 종료될 때까지 메모리를 점유하는 특성
	가독/쓰기 가능	- 실행 중 변수 값을 변경해야 하므로 Read-Write 권한이 부여됨
	프로세스 간 격리	- 동일 프로그램이라도 각 프로세스는 독립적인 데이터 영역을 가짐

- 프로그램 전체에서 접근 가능한 공통 데이터를 관리

II. 운영체제 메모리의 주요영역, 힙(Heap), 스택(Stack) 설명

가. 힙(Heap)의 설명

개념	- 프로그램 실행 도중(Runtime) 필요한 크기만큼 동적으로 할당받아 사용하는 자유 메모리 공간	
역할	동적 객체 저장	- 런타임에 결정되는 인스턴스, 배열 등 가변 크기 데이터를 위한 공간
	데이터 공유	- 서로 다른 스택 프레임(함수)에서 포인터를 통해 동일 데이터를 참조할 때 사용
	대용량 데이터 처리	- 스택의 한정된 크기를 넘어서는 대규모 자료구조(Tree, Graph 등) 관리
	영속적 데이터 관리	- 함수가 종료되어도 명시적으로 해제하기 전까지 데이터를 유지해야 할 때 활용
특징	Manual 제어	- 개발자(C/C++)나 가비지 컬렉터(Java/Python)가 할당 및 해제 주기를 관리
	Fragmentation	- 잦은 할당/해제로 인해 메모리 사이에 빈 공간이 생기는 단편화 문제 발생 가능
	Upward Growth	- 낮은 주소에서 높은 주소 방향으로 메모리를 할당하며 확장됨
	속도 상대적 저하	- 스택에 비해 할당/해제 시 오버헤드가 크며, 포인터 참조를 통한 간접 접근 필요

- 데이터의 크기를 미리 알 수 없는 경우(예: 가변 배열, 객체 생성) 사용

나. 스택(Stack)의 목적

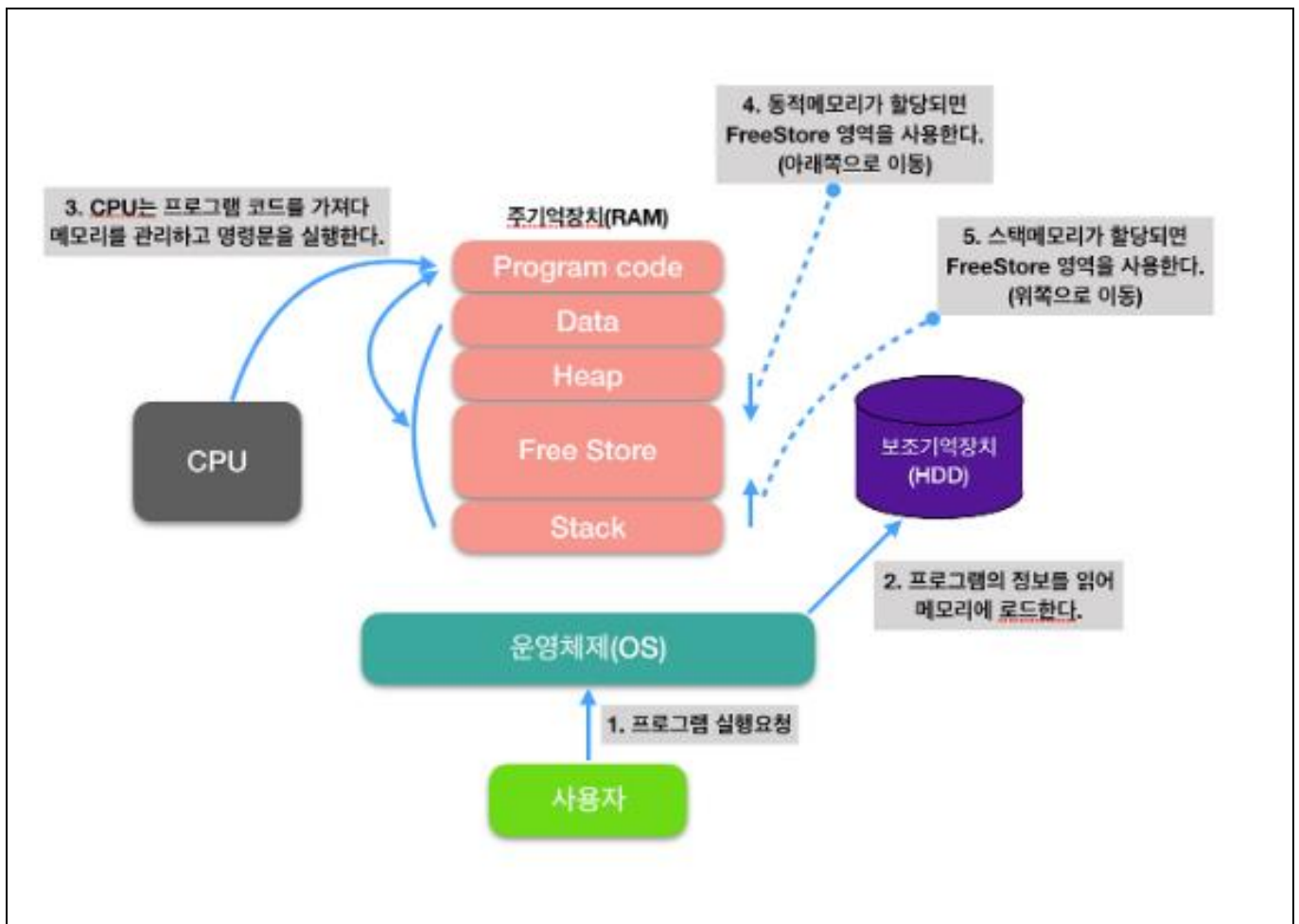
개념	- 함수 호출 시 발생하는 지역 변수, 매개 변수 및 복귀 주소 를 임시로 저장하는 공간	
역할	함수 컨텍스트 저장	- 함수 호출 시 필요한 매개변수, 지역변수, 복귀 주소를 하나의 프레임임으로 저장
	실행 순서 제어	- static 변수를 저장하여 함수가 종료되어도 값을 소멸시키지 않고 보존

	임시 연산 공간	- 수식 계산이나 단기적인 데이터 처리를 위한 임시 저장소 역할
	인터럽트 처리	- 시스템 인터럽트 발생 시 현재 수행 중인 작업을 백업하는 용도로 활용
특징	LIFO 구조	- 가장 마지막에 호출된 함수가 가장 먼저 종료되는 후입선출 논리 적용
	Automatic 관리	- 실행 중 변수 값을 변경해야 하므로 Read-Write 권한이 부여됨
	Downward Growth	- 높은 주소에서 낮은 주소 방향으로 확장되어 힙(Heap)과의 충돌 방지
	고속 접근	- CPU 레지스터(Stack Pointer)가 직접 주소를 관리하여 접근 속도가 매우 빠름

- 함수가 호출되면 할당되고 종료되면 즉시 해제되는 휘발성 영역

III. 영역 간 동작 메커니즘 및 설명

가. 영역 간 동작 메커니즘



- Code, Data, Heap, Stack 의 영역간 상호 메모리 동작

나. 영역 간 동작 메커니즘 설명

단계	주요 동작	설명
프로그램 실행 요청	사용자 → 운영체제(OS)	- 사용자가 프로그램을 실행하면 운영체제(OS)가 이를 감지하여 실행 준비를 시작
메모리 로드 (Loading)	보조기억장치 → 주기기억장치	- 보조기억장치(HDD/SSD)에 저장된 프로그램 정보를 주기기억장치(RAM)의 적절한 영역에 할당하여 올림
명령 인출 및 실행	주기기억장치 ↔ CPU	- CPU는 RAM의 'Program Code' 영역에서 명령어를 가져와 실행하며, 필요한 데이터를 다른 영역과 주고받음
동적 메모리 할당	Heap 영역 관리	- 프로그램 실행 중 발생하는 동적 데이터는 'Heap' 영역에 저장되며, 메모리 주소의 아래쪽 방향으로 할당
정적/임시 메모리 관리	Stack 영역 관리	- 함수 호출이나 지역 변수 등 임시 데이터는 'Stack' 영역에 저장되며, 메모리 주소의 위쪽 방향으로 할당

- 각 영역 간 동작을 통해 메모리 최적화 운영

“끝”



ITPE 기술사회

제138회 정보처리기술사 기출문제 해설집

대 상	정보관리기술사, 컴퓨터시스템응용기술사, 정보통신기술사, 정보시스템감리사 시험
발행일	2026년 02월 07일
집 필	강정배PE, 정상PE, 전일PE, 백현PE, 조종흥PE
출 판	ITPE(Information Technology Professional Engineer)
주 소	ITPE 대치점 서울시 강남구 선릉로 86길 17 선릉엠티빌딩 7층 ITPE 선릉점 서울시 강남구 선릉로 86길 15, 3층 IT교육센터 아이티피이 ITPE 강남점 서울시 강남구 테헤란로 52길 21 파라다이스벤처타워 3층 303호 ITPE 영등포점 서울시 영등포구 당산동2가 하나비즈타워 7층 ITPE ITPE 을지로점 서울시 중구 삼일대로 363, 2615호(장교동 장교빌딩)
연락처	070-4077-1267 / itpe@itpe.co.kr

본 저작물은 [ITPE\(아이티피이\)](#)에 저작권이 있습니다.

저작권자의 허락없이 **본 저작물을 불법적인 복제 및 유통, 배포**하는 경우
법적인 처벌을 받을 수 있습니다.